لیست‌ها، دیکشنری‌ها و سایر ساختارهای داده در پایتون

در برنامه‌های واقعی، معمولاً با یک مقدار ساده سروکار نداریم؛ ممکن است لازم باشد فهرستی از محصولات، اطلاعات چند کاربر یا مجموعه‌ای از نتایج را نگهداری کنیم.

برای مدیریت چنین داده‌هایی، پایتون ساختارهای داده مختلفی دارد. در این بخش با چهار ساختار پرکاربرد آشنا می‌شویم: لیست (`list`)، تاپل (`tuple`)، دیکشنری (`dict`) و مجموعه (`set`).

۱. لیست (List)

لیست برای نگهداری چند مقدار در یک مجموعه استفاده می‌شود. مقادیر داخل لیست می‌توانند تکراری باشند و ترتیب آن‌ها حفظ می‌شود.

برای ساخت لیست، از براکت‌های مربعی [] استفاده می‌کنیم:

```
fruits = ["apple", "banana", "orange"]

print(fruits)
```

خروجی:

```
['apple', 'banana', 'orange']
```

دسترسی به عناصر لیست

هر عنصر در لیست یک موقعیت یا اندیس دارد. در پایتون، اندیس‌گذاری از صفر شروع می‌شود.

```
fruits = ["apple", "banana", "orange"]

print(fruits[0])
print(fruits[1])
print(fruits[2])
```

خروجی:

```
apple
banana
orange
```

برای دسترسی به آخرین عنصر نیز می‌توانیم از اندیس -1 استفاده کنیم:

```
print(fruits[-1])
```

خروجی:

```
orange
```

اضافه‌کردن و حذف عناصر

لیست قابل تغییر است؛ بنابراین می‌توانیم عناصر آن را اضافه، حذف یا جایگزین کنیم.

```
fruits = ["apple", "banana"]

fruits.append("orange")
print(fruits)

fruits.remove("banana")
print(fruits)
```

خروجی:

```
['apple', 'banana', 'orange']
['apple', 'orange']
```

متد `append()` یک عنصر را به انتهای لیست اضافه می‌کند و `remove()` اولین عنصر مطابق با مقدار داده‌شده را حذف می‌کند. اگر مقدار موردنظر در لیست نباشد، `remove()` خطا ایجاد می‌کند.

برای تغییر یک عنصر نیز می‌توانیم از اندیس آن استفاده کنیم:

```
fruits = ["apple", "banana", "orange"]

fruits[1] = "grape"

print(fruits)
```

خروجی:

```
['apple', 'grape', 'orange']
```

تعداد عناصر لیست

تابع `len()` تعداد عناصر یک لیست را برمی‌گرداند.

```
fruits = ["apple", "banana", "orange"]

print(len(fruits))
```

خروجی:

```
3
```

کاربرد لیست: نگهداری فهرست محصولات، نام دانشجویان، وظایف روزانه یا نتایج یک محاسبه.

۲. پیمایش لیست با حلقه

بسیاری از مواقع می‌خواهیم عملیاتی را روی تمام عناصر یک لیست انجام دهیم. برای این کار می‌توانیم از حلقه `for` استفاده کنیم.

```
fruits = ["apple", "banana", "orange"]

for fruit in fruits:
    print(fruit)
```

خروجی:

```
apple
banana
orange
```

در هر دور حلقه، متغیر `fruit` یکی از عناصر لیست را دریافت می‌کند.

برای مثال، می‌توانیم قیمت چند محصول را با هم جمع کنیم:

```
prices = [100, 250, 150]

total = 0

for price in prices:
    total += price

print(total)
```

خروجی:

```
500
```

عبارت `price += total` معادل `price + total = total` است. به این ترتیب، قیمت هر محصول به مجموع قبلی اضافه می‌شود.

۳. تاپل (Tuple)

تاپل نیز مانند لیست می‌تواند چند مقدار را به‌ترتیب نگهداری کند، اما پس از ساخته‌شدن، عناصر آن قابل جایگزینی، اضافه یا حذف نیستند.

برای ساخت تاپل معمولاً از پرانتز استفاده می‌کنیم:

```
coordinates = (10, 20)

print(coordinates[0])
print(coordinates[1])
```

خروجی:

```
10
20
```

اما تغییر یک عنصر تاپل مجاز نیست:

```
coordinates = (10, 20)

coordinates[0] = 15  # This raises an error
```

تاپل زمانی مفید است که بخواهیم چند مقدار مرتبط را کنار هم نگه داریم و نخواهیم عناصر آن‌ها در طول برنامه تغییر کنند.

برای مثال:

```
screen_size = (1920, 1080)
```

در این مثال، دو مقدار می‌توانند نشان‌دهنده عرض و ارتفاع یک صفحه باشند.

نکته: اگر تاپل فقط یک عنصر داشته باشد، باید بعد از آن ویرگول قرار دهیم:

```
single_value = (10,)
```

بدون ویرگول، عبارت `(10)` صرفاً یک عدد داخل پرانتز است، نه یک تاپل.

۴. دیکشنری (Dictionary)

گاهی به‌جای دسترسی به داده‌ها با شماره موقعیت، می‌خواهیم هر مقدار را با یک نام یا کلید مشخص کنیم.

دیکشنری برای نگهداری داده‌ها به‌صورت کلید و مقدار استفاده می‌شود.

برای نمونه، اطلاعات یک کاربر را در نظر بگیرید:

```
user = {
    "name": "Sara",
    "age": 25,
    "city": "Berlin"
}
```

در این دیکشنری، کلیدهای `name`، `age` و `city` به‌ترتیب به نام، سن و شهر کاربر اشاره می‌کنند.

دسترسی به مقادیر

برای دریافت یک مقدار، کلید مربوط به آن را به کار می‌بریم:

```
print(user["name"])
print(user["age"])
```

خروجی:

```
Sara
25
```

اگر کلیدی وجود نداشته باشد، دسترسی مستقیم با `user["key"]` خطا ایجاد می‌کند. برای دسترسی امن‌تر می‌توانیم از متد `get()` استفاده کنیم:

```
print(user.get("email"))
print(user.get("email", "Not provided"))
```

خروجی:

```
None
Not provided
```

در مثال دوم، اگر کلید `email` وجود نداشته باشد، مقدار پیش‌فرض `"Not provided"` برگردانده می‌شود.

اضافه‌کردن و تغییر اطلاعات

برای اضافه‌کردن یک مقدار جدید یا تغییر مقدار موجود، کافی است آن را به کلید موردنظر نسبت دهیم:

```
user = {
    "name": "Sara",
    "age": 25
}

user["city"] = "Berlin"
user["age"] = 26

print(user)
```

خروجی:

```
{'name': 'Sara', 'age': 26, 'city': 'Berlin'}
```

پیمایش دیکشنری

با استفاده از `items()` می‌توانیم کلید و مقدار هر ورودی را دریافت کنیم:

```
user = {
    "name": "Sara",
    "age": 25,
    "city": "Berlin"
}

for key, value in user.items():
    print(key, value)
```

خروجی:

```
name Sara
age 25
city Berlin
```

کاربرد دیکشنری: نگهداری اطلاعات کاربران، مشخصات محصولات، تنظیمات برنامه یا داده‌هایی که هر بخش آن با یک کلید شناخته می‌شود.

۵. مجموعه (Set)

مجموعه برای نگهداری مقادیر یکتا استفاده می‌شود؛ یعنی هر مقدار فقط یک بار در مجموعه حضور دارد. ترتیب عناصر نیز مبنای مناسبی برای دسترسی با اندیس نیست.

برای ساخت مجموعه از آکولاد استفاده می‌کنیم:

```
numbers = {1, 2, 3, 2, 1}

print(numbers)
```

خروجی، مجموعه‌ای شامل 1، 2 و 3 خواهد بود. مقادیر تکراری حذف می‌شوند. ترتیب نمایش عناصر مجموعه تضمین‌شده نیست.

اضافه‌کردن و حذف عناصر

```
numbers = {1, 2, 3}

numbers.add(4)
numbers.discard(2)

print(numbers)
```

خروجی، مجموعه‌ای شامل 1، 3 و 4 خواهد بود.

متد `add()` مقدار جدیدی اضافه می‌کند و `discard()` مقدار مشخص‌شده را در صورت وجود حذف می‌کند. اگر آن مقدار وجود نداشته باشد، `discard()` خطا ایجاد نمی‌کند.

پیدا کردن مقادیر مشترک

مجموعه‌ها برای بررسی اشتراک بین داده‌ها نیز مفیدند:

```
students_a = {"Ali", "Sara", "Reza"}
students_b = {"Sara", "Reza", "Mina"}

common_students = students_a & students_b

print(common_students)
```

خروجی شامل `Sara` و `Reza` خواهد بود؛ یعنی افرادی که در هر دو مجموعه حضور دارند.

کاربرد مجموعه: حذف داده‌های تکراری، بررسی عضویت و پیدا کردن مقادیر مشترک بین دو مجموعه داده.

برای ساخت مجموعه خالی باید از `set()` استفاده کنیم، چون `{}` یک دیکشنری خالی می‌سازد:

```
empty_set = set()
empty_dictionary = {}
```

۶. مقایسه ساختارهای داده

هر ساختار داده برای نوع خاصی از نیازها مناسب‌تر است:

ساختار	اصلی ویژگی	نمونه کاربرد
`list`	تکرار امکان دارای و تغییر قابل مرتب،	محصولات فهرست
`tuple`	تغییر غیرقابل و مرتب	ثابت مقادیر یا مختصات
`dict`	مقدار و کلید صورت‌به داده نگهداری	کاربر یک اطلاعات
`set`	یکتا مقادیر نگهداری	تکراری موارد حذف

برای انتخاب ساختار مناسب، باید به نیاز برنامه توجه کنیم. اگر ترتیب عناصر و امکان تغییر آن‌ها مهم باشد، لیست انتخاب مناسبی است. اگر بخواهیم به هر مقدار یک کلید اختصاص دهیم، معمولاً دیکشنری مناسب‌تر است. اگر یکتا بودن مقادیر اهمیت داشته باشد، می‌توانیم از مجموعه استفاده کنیم.

۷. ترکیب ساختارهای داده

در برنامه‌های واقعی، معمولاً این ساختارها را در کنار هم به کار می‌بریم.

برای مثال، فرض کنید می‌خواهیم اطلاعات چند محصول را نگهداری کنیم. هر محصول چند ویژگی دارد و برای همه محصولات به یک فهرست نیاز داریم.

```
products = [
    {"name": "Laptop", "price": 1200},
    {"name": "Mouse", "price": 25},
    {"name": "Keyboard", "price": 75}
]

for product in products:
    print(product["name"], product["price"])
```

خروجی:

```
Laptop 1200
Mouse 25
Keyboard 75
```

در این مثال:

- یک لیست داریم که چند محصول را نگهداری می‌کند.
- هر محصول یک دیکشنری است.
- دیکشنری هر محصول، نام و قیمت آن را نگه می‌دارد.
- حلقه `for` روی محصولات حرکت می‌کند و اطلاعات هر کدام را نمایش می‌دهد.

این روش در بسیاری از برنامه‌ها کاربرد دارد. برای مثال، پاسخ یک سرویس وب ممکن است شامل فهرستی از دیکشنری‌ها باشد که اطلاعات کاربران یا محصولات را نگهداری می‌کنند.

۸. ساختارهای داده و هوش مصنوعی

ابزارهای هوش مصنوعی هنگام تولید کد، مرتب از لیست‌ها، دیکشنری‌ها و سایر ساختارهای داده استفاده می‌کنند. به همین دلیل، شناخت آن‌ها به ما کمک می‌کند کد تولیدشده را بهتر بخوانیم و بررسی کنیم.

برای مثال، اگر از یک ابزار هوش مصنوعی بخواهیم برنامه‌ای برای مدیریت محصولات بسازد، ممکن است برای نگهداری تمام محصولات از لیست و برای مشخصات هر محصول از دیکشنری استفاده کند.

ما باید بتوانیم تشخیص دهیم:

- داده‌ها چگونه سازمان‌دهی شده‌اند؟
- برای دسترسی به هر مقدار از اندیس استفاده می‌شود یا کلید؟
- آیا داده‌های تکراری مجازند؟
- آیا تغییر عناصر در طول اجرای برنامه لازم است؟

قرار نیست همه ساختارها را از ابتدا به‌طور کامل بشناسیم. کافی است کاربرد اصلی آن‌ها را درک کنیم و به‌تدریج با استفاده از مثال‌های عملی مهارت بیشتری به دست آوریم.

جمع‌بندی

ساختارهای داده به ما کمک می‌کنند اطلاعات برنامه را به شکلی مناسب سازمان‌دهی کنیم. لیست برای مجموعه‌های مرتب و قابل تغییر، تاپل برای داده‌های مرتب و غیرقابل تغییر، دیکشنری برای ارتباط کلید و مقدار و مجموعه برای نگهداری مقادیر یکتا کاربرد دارد.

انتخاب ساختار مناسب، خوانایی و سادگی برنامه را افزایش می‌دهد و درک برنامه‌هایی را که با کمک هوش مصنوعی تولید شده‌اند آسان‌تر می‌کند.

تمرین

برنامه‌ای بنویسید که اطلاعات سه محصول را نگهداری کند. برای هر محصول، نام و قیمت را ذخیره کنید و سپس نام و قیمت همه محصولات را نمایش دهید.

راهنمای تمرین:

1. یک لیست به نام `products` بسازید.
2. هر محصول را به‌صورت یک دیکشنری با کلیدهای `name` و `price` تعریف کنید.
3. با استفاده از حلقه `for` روی لیست حرکت کنید.
4. نام و قیمت هر محصول را نمایش دهید.
5. در پایان، تعداد محصولات را نیز با استفاده از `len()` نمایش دهید.